device.py — technical

src/moe_pruning/utils/device.py · 83 lines · moe_pruning.utils.device

Purpose

Device and dtype helpers.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class DeviceUnavailableError`

lines 18-19 · bases RuntimeError

Raised when an explicitly requested device is not available.

`is_available`

```
def is_available(kind: str) -> bool
```

lines 22-29 · raises ValueError

True if a device type (cpu / cuda / mps) can be used.

`resolve_device`

```
def resolve_device(requested: str='auto') -> torch.device
```

lines 32-51 · raises DeviceUnavailableError, ValueError

Resolve a device request.

1. `auto`: CUDA if available, else MPS, else CPU. Explicit choices are validated; unavailable -> `DeviceUnavailableError` (no fallback).

`resolve_dtype`

```
def resolve_dtype(name: str) -> torch.dtype
```

lines 54-58 · raises ValueError

Map MODEL_DTYPE to a torch dtype.

`check_dtype_supported`

```
def check_dtype_supported(device: torch.device, dtype: torch.dtype) -> None
```

lines 61-65

Warn about poor device/dtype combinations.

`log_device`

```
def log_device(device: torch.device, dtype: torch.dtype) -> None
```

lines 68-69

Log the selected device and dtype.

`synchronize`

```
def synchronize(device: torch.device) -> None
```

lines 72-76

Wait for pending GPU work.

`empty_cache`

```
def empty_cache(device: torch.device) -> None
```

lines 79-83

Release cached GPU memory.

Module constants

Name	Value
`_DTYPES`	{'float32': torch.float32, 'float16': torch.float16, 'bfloat16': torch.bfloat16}

Errors raised

Function	Exception	Message (f-string fields in braces)
`is_available`	ValueError	Unknown device type {kind}
`resolve_device`	ValueError	DEVICE must be auto, cuda, mps or cpu (got {requested})
`resolve_device`	DeviceUnavailableError	DEVICE={requested} was requested but is not available on this machine. Set DEVICE=auto or choose an available device.
`resolve_dtype`	ValueError	MODEL_DTYPE must be one of {list(_DTYPES)} (got {name})